

여행 앱 백엔드 구현 계획서

FastAPI + Supabase(Auth/DB) + Vercel 배포 기준 · AI 응답 캐싱 설계 포함

1. 기술 스택 개요

영역	선택
프레임워크	Python + FastAPI
ORM / 검증	SQLAlchemy + Pydantic
DB / Auth	Supabase (PostgreSQL + Supabase Auth)
인증 검증	JWKS 기반 JWT 검증 (PyJWT)
AI	Anthropic Python SDK (스트리밍)
캐싱	Upstash Redis (서버리스 REST 기반)
배포	Vercel (프론트 + 백엔드 모노레포, 서버리스 함수)
로컬 개발	Docker + docker - compose (선택, 팀 환경 통일용)

2. 인증 — Supabase Auth 전환

자체 JWT 발급(python - jose + passlib) 방식을 폐기하고, Supabase Auth가 발급한 JWT를 백엔드에서 검증만 하는 구조로 전환한다. RLS는 사용하지 않고, 권한 체크는 애플리케이션(서비스) 레이어에서 처리한다.

2-1. 변경 사항

- api/auth.py 제거 — 로그인/가입/비밀번호 재설정 엔드포인트 불필요. 클라이언트가 Supabase SDK로 직접 처리.
- models/user.py → models/profile.py — auth.users.id를 참조하는 프로필 확장 테이블로 역할 변경.
- middleware/auth.py → dependencies/auth.py — FastAPI Dependency 패턴으로 전환, JWKS 기반 검증.
- Postgres 트리거 — auth.users에 신규 유저 생성 시 profiles row 자동 생성 (마이그레이션에 포함).
- requirements.txt — passlib, python - jose 제거 → pyjwt[crypto] 추가.

2-2. JWT 검증 (dependencies/auth.py)

```
import jwt
from jwt import PyJWKClient
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

security = HTTPBearer()
jwks_client = PyJWKClient(f'{settings.SUPABASE_URL}/auth/v1/.well-known/jwks.json')

def get_current_user(creds: HTTPAuthorizationCredentials = Depends(security)) -> str:
```

```

token = creds.credentials
try:
    signing_key = jwks_client.get_signing_key_from_jwt(token)
    payload = jwt.decode(
        token, signing_key.key,
        algorithms=["ES256", "RS256"], audience="authenticated",
    )
    return payload["sub"] # profiles.id
except jwt.PyJWTError:
    raise HTTPException(status_code=401, detail="Invalid or expired token")

```

* 프로젝트가 레거시(HS256 공유 시크릿) 상태라면 SUPABASE_JWT_SECRET으로 더 단순하게 검증 가능.
Supabase 대시보드 → Auth → JWT Keys에서 현재 서명 방식 확인 필요.

2-3. 신규 유저 프로필 자동 생성 트리거

```

create function public.handle_new_user()
returns trigger as $$
begin
    insert into public.profiles (id, email)
    values (new.id, new.email);
    return new;
end;
$$ language plpgsql security definer;

create trigger on_auth_user_created
after insert on auth.users
for each row execute procedure public.handle_new_user();

```

3. 최종 폴더 구조 (Supabase Auth 기준)

```
backend/
├── src/
│   ├── main.py
│   ├── config/
│   │   ├── database.py          # Supabase Pooler 연결 (port 6543)
│   │   └── settings.py          # SUPABASE_URL, DATABASE_URL 등
│   ├── api/
│   │   ├── trips.py
│   │   └── baggage.py
│   ├── models/
│   │   ├── profile.py           # auth.users.id 참조
│   │   ├── trip.py
│   │   ├── item.py
│   │   └── baggage_rule.py
│   ├── schemas/
│   │   ├── trip.py
│   │   ├── baggage.py
│   │   └── profile.py
│   ├── services/
│   │   ├── ai_service.py        # 스트리밍 + 캐싱 로직
│   │   ├── baggage_service.py
│   │   ├── cache_service.py     # Upstash Redis 래퍼 (신규)
│   │   └── trip_service.py
│   ├── dependencies/
│   │   └── auth.py              # JWKS 검증
│   └── utils/
│       └── normalizer.py        # 캐시 키용 제품명/항공사명 정규화
├── migrations/versions/
│   └── xxxx_add_profile_trigger.py
├── requirements.txt
├── Dockerfile                  # 로컬 개발 전용 (Vercel 배포엔 미사용)
├── docker-compose.yml          # 로컬 개발 전용
├── vercel.json                 # Vercel 라우팅/빌드 설정
└── pyproject.toml
```

4. 배포 아키텍처 — Vercel

Vercel은 커스텀 Dockerfile을 지원하지 않는다. FastAPI 앱은 requirements.txt 기반으로 Vercel이 자체적으로 서버리스 함수(Fluid Compute)로 빌드한다. Docker는 팀 로컬 개발 환경 통일용으로만 유지하고, 배포 파이프라인과는 분리된 트랙으로 취급한다.

- 프론트(Next.js 등)와 백엔드(FastAPI)를 하나의 Vercel 프로젝트(모노레포)로 서빙 가능.
- /api/* → FastAPI 백엔드, /* → 프론트, vercel.json에서 라우팅 통합.
- 같은 도메인이라 CORS 설정 부담이 거의 없음.
- git push 한 번으로 프론트 + 백엔드 동시 배포, 브랜치별 프리뷰 배포도 함께 생성.

vercel.json 예시

```
{
  "builds": [
    { "src": "frontend/package.json", "use": "@vercel/next" },
    { "src": "backend/src/main.py", "use": "@vercel/python" }
  ],
  "routes": [
```

```

    { "src": "/api/(.*)", "dest": "backend/src/main.py" },
    { "src": "/(.*)", "dest": "frontend/$1" }
  ]
}

```

5. 서버리스 환경 제약과 대응

항목	제약	대응
실행 시간	Free 10초 / Pro 60초 타임아웃	AI 응답 스트리밍(SSE)으로 체감 대기시간 최소화
백그라운드 워커	큐/워커(Celery 등) 미지원	현 단계 불필요. 필요 시 Railway 등으로 워커만 분리
캐싱	in-memory 캐시 무의미 (함수 인스턴스 매번 재생성 가능)	Upstash Redis(REST 기반, 서버리스 전용)로 대체
DB 커넥션	함수마다 새 커넥션 → 풀 고갈 위험	Supabase Pooler(PgBouncer, 6543) + NullPool 사용

5-1. DB 커넥션 설정

```

# config/database.py
from sqlalchemy import create_engine
from sqlalchemy.pool import NullPool

# Supabase Transaction Pooler (port 6543) 사용 — direct(5432) 금지
DATABASE_URL = settings.DATABASE_URL # ...pooler.supabase.com:6543/postgres

engine = create_engine(DATABASE_URL, poolclass=NullPool, pool_pre_ping=True)

```

5-2. AI 응답 스트리밍

```

# api/trips.py
from fastapi.responses import StreamingResponse

@router.post("/trips/generate")
async def generate_trip(req: TripRequest):
    async def event_stream():
        async with anthropic_client.messages.stream(...) as stream:
            async for text in stream.text_stream:
                yield f"data: {text}\n\n"
    return StreamingResponse(event_stream(), media_type="text/event-stream")

```

6. AI 응답 캐싱 설계 (수화물 판정)

동일한 '항공사 + 제품' 조합에 대한 수화물 판정 질의가 반복될 가능성이 높으므로, AI 판정 결과를 Upstash Redis에 캐싱하여 Anthropic API 비용 절감과 응답 속도 개선을 동시에 달성한다.

6-1. 캐시 플로우

- ① 유저가 '항공사 + 제품명'으로 수화물 판정 요청
- ② `utils/normalizer.py`로 항공사명 · 제품명을 정규화 (대소문자, 공백, 오타자 등 표준화)
- ③ 정규화된 값으로 캐시 키 생성 (예: `baggage:koreanair:samsung - galaxy - tab - s9`)
- ④ Redis 조회 → 있으면 즉시 반환 (Anthropic API 호출 없음)
- ⑤ 없으면 `baggage_service`의 규칙 엔진 우선 판정 → 애매한 경우만 AI 보조 호출
- ⑥ AI 판정 결과를 Redis에 TTL과 함께 저장 후 반환

6-2. 캐시 키 설계 원칙

- 정규화가 핵심: '대한항공' / 'Korean Air' / 'KE'가 모두 같은 키로 매핑되어야 캐시 적중률이 올라감.
- 네임스페이스 분리: `baggage:ai:{airline}:{product}` 형태로 프리픽스를 뒤서 추후 다른 캐시 용도와 충돌 방지.
- 버전 포함 고려: 판정 로직/프롬프트가 바뀌면 캐시가 무효화되도록 `v1`: 같은 버전 태그를 키에 포함.

6-3. `services/cache_service.py`

```
import httpx
from src.config.settings import settings

UPSTASH_URL = settings.UPSTASH_REDIS_URL
UPSTASH_TOKEN = settings.UPSTASH_REDIS_TOKEN
HEADERS = {"Authorization": f"Bearer {UPSTASH_TOKEN}"}
```

```
async def get_cached(key: str) -> str | None:
    async with httpx.AsyncClient() as client:
        resp = await client.get(f"{UPSTASH_URL}/get/{key}", headers=HEADERS)
        data = resp.json()
        return data.get("result")

async def set_cached(key: str, value: str, ttl_seconds: int = 60 * 60 * 24 * 30):
    async with httpx.AsyncClient() as client:
        await client.post(
            f"{UPSTASH_URL}/set/{key}",
            headers=HEADERS,
            json={"value": value, "EX": ttl_seconds},
        )
```

6-4. `ai_service.py` 캐시 통합

```
from src.services import cache_service
from src.utils.normalizer import normalize_airline, normalize_product

async def get_baggage_verdict(airline: str, product: str) -> dict:
    key = f"baggage:ai:v1:{normalize_airline(airline)}:{normalize_product(product)}"

    cached = await cache_service.get_cached(key)
    if cached:
```

```
return json.loads(cached)    # cache hit — Anthropic API 미호출

verdict = await call_anthropic_for_verdict(airline, product)    # cache miss
await cache_service.set_cached(key, json.dumps(verdict))
return verdict
```

6-5. TTL 및 무효화 전략

- 기본 TTL 30일 — 항공사 수화물 규정은 자주 바뀌지 않으므로 긴 TTL이 합리적.
- 규정 변경 시 수동 무효화 — baggage_rule 테이블 업데이트 시 관련 캐시 키를 함께 삭제하는 관리자 엔드포인트 고려.
- 부분 매치 방지 — 제품명이 애매하게 다른 경우(용량, 모델명 등) 오답 캐시가 재사용되지 않도록 normalizer에서 핵심 식별자만 남기고 나머지는 버림.

* 캐시 미스 시 규칙 엔진(baggage_service)이 먼저 판정하고, AI는 애매한 케이스에만 보조로 호출하는 구조를 유지하면 캐싱 효과와 별개로 API 호출 자체도 줄어든다.

7. 다음 단계 체크리스트

- dependencies/auth.py — JWKS 검증 구현
- profiles 테이블 + auth 트리거 마이그레이션 작성
- config/database.py — Supabase Pooler(6543) 연결로 전환
- ai_service.py — 스트리밍 응답 구현
- cache_service.py — Upstash Redis 연동
- normalizer.py — 항공사/제품명 정규화 로직 (캐시 키 설계 핵심)
- vercel.json — 모노레포 라우팅 설정
- (선택) docker - compose.yml — 팀 로컬 개발 환경

이 문서는 대화 내용을 기반으로 정리된 구현 계획입니다. 실제 구현 시 세부 사항은 변경될 수 있습니다.